

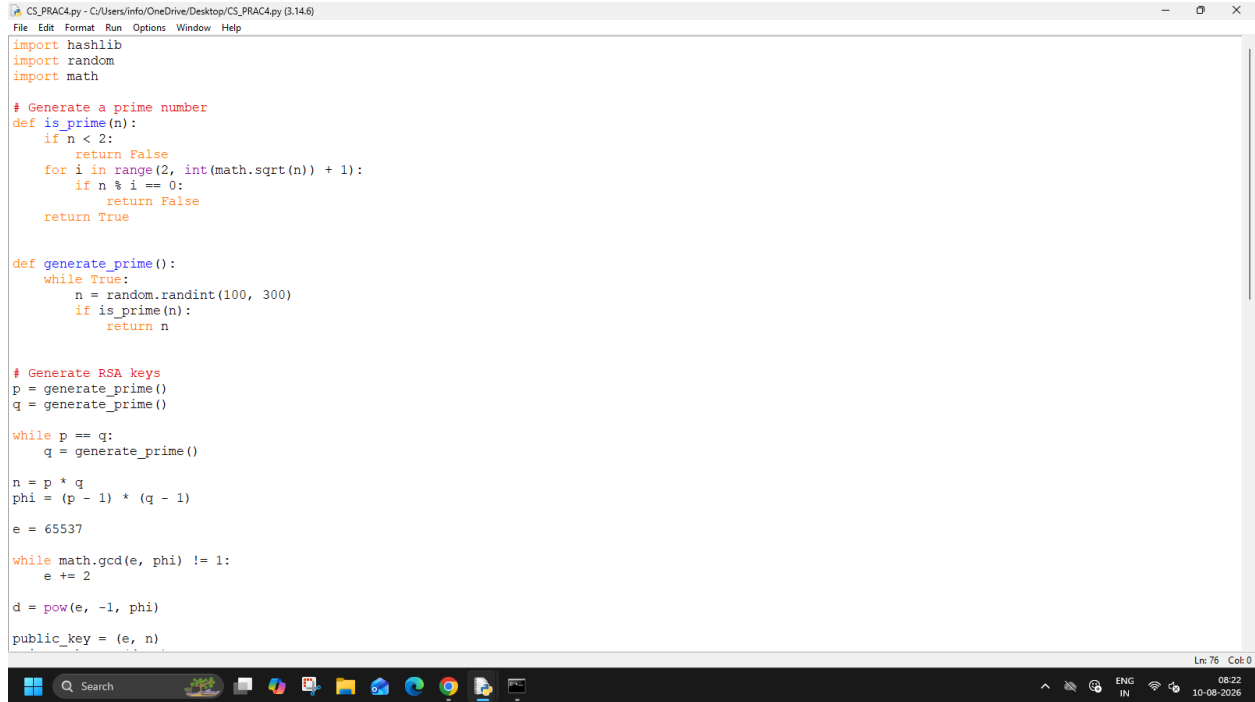
SHETH L.U.J AND SIR M.V. COLLEGE
SUBJECT: CYBER & INFORMATION SECURITY

PRACTICAL - 4

AIM: Implement digital signature algorithms such as RSA-based signatures, and verify the integrity and authenticity of digitally signed messages.

CLI:

Code:



```
CS_PRAC4.py - C:/Users/info/OneDrive/Desktop/CS_PRAC4.py (3.14.6)
File Edit Format Run Options Window Help

import hashlib
import random
import math

# Generate a prime number
def is_prime(n):
    if n < 2:
        return False
    for i in range(2, int(math.sqrt(n)) + 1):
        if n % i == 0:
            return False
    return True

def generate_prime():
    while True:
        n = random.randint(100, 300)
        if is_prime(n):
            return n

# Generate RSA keys
p = generate_prime()
q = generate_prime()

while p == q:
    q = generate_prime()

n = p * q
phi = (p - 1) * (q - 1)

e = 65537

while math.gcd(e, phi) != 1:
    e += 2

d = pow(e, -1, phi)

public_key = (e, n)
```

Ln: 76 Col: 0

Windows taskbar at the bottom shows the Start button, Search bar, and various application icons. The system tray on the right indicates the language is ENG, the input is IN, and the date and time are 08:22 on 10-08-2026.

SHETH L.U.J AND SIR M.V. COLLEGE
SUBJECT: CYBER & INFORMATION SECURITY

```
CS_PRAC4.py - C:/Users/info/OneDrive/Desktop/CS_PRAC4.py (3.14.6)
File Edit Format Run Options Window Help

d = pow(e, -1, phi)
public_key = (e, n)
private_key = (d, n)

print("RSA Keys Generated")
print("Public Key :", public_key)
print("Private Key:", private_key)

# Get message
message = input("\nEnter message: ")

# Hash the message
hash_value = hashlib.sha256(message.encode()).hexdigest()

# Convert hash to integer
hash_int = int(hash_value, 16)

# Reduce hash to fit small demonstration RSA key
hash_int = hash_int % n

# Create digital signature using private key
signature = pow(hash_int, d, n)

print("\nSHA-256 Hash:", hash_value)
print("Digital Signature:", signature)

# Verify signature
received_hash = pow(signature, e, n)

current_hash = int(
    hashlib.sha256(message.encode()).hexdigest(),
    16
) % n

Ln 76 Col 0
```

```
CS_PRAC4.py - C:/Users/info/OneDrive/Desktop/CS_PRAC4.py (3.14.6)
File Edit Format Run Options Window Help

hash_int = hash_int % n

# Create digital signature using private key
signature = pow(hash_int, d, n)

print("\nSHA-256 Hash:", hash_value)
print("Digital Signature:", signature)

# Verify signature
received_hash = pow(signature, e, n)

current_hash = int(
    hashlib.sha256(message.encode()).hexdigest(),
    16
) % n

if received_hash == current_hash:
    print("\nSignature Verification: SUCCESS")
    print("Message is authentic and has not been modified.")
else:
    print("\nSignature Verification: FAILED")

# Test modified message
modified_message = input("\nEnter modified message: ")

modified_hash = int(
    hashlib.sha256(modified_message.encode()).hexdigest(),
    16
) % n

if received_hash == modified_hash:
    print("Modified message: VALID")
else:
    print("Modified message: INVALID")
    print("Integrity check failed.")

Ln 76 Col 0
```

SHETH L.U.J AND SIR M.V. COLLEGE

SUBJECT:CYBER & INFORMATION SECURITY

OUTPUT:

```
IDLE Shell 3.14.6
File Edit Shell Debug Options Window Help
Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/info/OneDrive/Desktop/CS_PRAC4.py =====
RSA Keys Generated
Public Key : (65537, 50689)
Private Key: (31201, 50689)

Enter message: T125

SHA-256 Hash: fa60808ab2313417f03ffaab155bc577637a8915c2443e0a7d1be180291c3284
Digital Signature: 5672

Signature Verification: SUCCESS
Message is authentic and has not been modified.

Enter modified message: T125
Modified message: VALID
>>>
```

```
C:\WINDOWS\system32\cmd.exe
Microsoft Windows [Version 10.0.26200.8875]
(c) Microsoft Corporation. All rights reserved.

C:\Users\info>python "C:\Users\info\OneDrive\Desktop\CS_PRAC4.py"
RSA Keys Generated
Public Key : (65537, 48959)
Private Key: (41177, 48959)

Enter message: T125

SHA-256 Hash: fa60808ab2313417f03ffaab155bc577637a8915c2443e0a7d1be180291c3284
Digital Signature: 26605

Signature Verification: SUCCESS
Message is authentic and has not been modified.

Enter modified message: T125
Modified message: VALID

C:\Users\info>
```

SUMEET YADAV
T125

SHETH L.U.J AND SIR M.V. COLLEGE
SUBJECT: CYBER & INFORMATION SECURITY

GUI:

Code:

```
*cs_prac3_gui.py - C:/Users/info/OneDrive/Desktop/cs_prac3_gui.py (3.14.6)
File Edit Format Run Options Window Help
import tkinter as tk
from tkinter import messagebox
import hashlib
import random
import math

#RSA FUNCTIONS
def is_prime(n):
    if n < 2:
        return False

    for i in range(2, int(math.sqrt(n)) + 1):
        if n % i == 0:
            return False
    return True

def generate_prime():
    while True:
        number = random.randint(100, 300)

        if is_prime(number):
            return number

def generate_keys():
    global public_key, private_key

    p = generate_prime()
    q = generate_prime()
    while p == q:
        q = generate_prime()
    n = p * q
    phi = (p - 1) * (q - 1)
    e = 65537
    while math.gcd(e, phi) != 1:
        e += 2
    d = pow(e, -1, phi)
    public_key = (e, n)
    private_key = (d, n)
    public_key_label.config(
        text=f"Public Key: {public_key}, Private Key: {private_key}"
    )

def main():
    generate_keys()

    # GUI Setup
    root = tk.Tk()
    root.title("RSA Key Generation")
    root.geometry("400x200")

    status_label = tk.Label(root, text="RSA keys generated successfully.", fg="green")
    status_label.pack(pady=10)

    # Hash and Sign Functions
    def get_hash(message, n):
        hash_value = hashlib.sha256(
            message.encode()
        ).hexdigest()
        hash_integer = int(hash_value, 16)
        return hash_value, hash_integer % n

    def sign_message():
        if not public_key or not private_key:
            messagebox.showwarning(
                "Warning",
                "Please generate RSA keys first."
            )
            return
        message = message_text.get("1.0", tk.END).strip()
        if not message:
            messagebox.showwarning(
                "Warning",
                "Please enter a message."
            )
            return

        d, n = private_key
        hash_value, hash_integer = get_hash(message, n)
        signature = pow(hash_integer, d, n)
        signature_entry.delete(0, tk.END)
        signature_entry.insert(0, hash_value)
        signature_entry.insert(0, str(signature))

        status_label.config(
            text="Digital signature generated successfully.",
            fg="green"
        )

    # GUI Elements
    message_text = tk.Text(root, height=5)
    message_text.pack(pady=5)

    signature_entry = tk.Entry(root)
    signature_entry.pack(pady=5)

    sign_button = tk.Button(
        root,
        text="Sign Message",
        command=sign_message
    )
    sign_button.pack(pady=5)

    root.mainloop()

if __name__ == "__main__":
    main()
```

SUMEET YADAV
T125

SHETH L.U.J AND SIR M.V. COLLEGE

SUBJECT:CYBER & INFORMATION SECURITY

```
*cs_prac3_gui.py - C:/Users/info/OneDrive/Desktop/cs_prac3_gui.py (3.14.6)
File Edit Format Run Options Window Help

    status_label.config(
        text="Digital signature generated successfully.",
        fg="green"
    )

def verify_signature():
    if not public_key:
        messagebox.showwarning(
            "Warning",
            "Please generate RSA keys first."
        )
        return

    message = message_text.get("1.0", tk.END).strip()
    signature_text = signature_entry.get().strip()
    if not message or not signature_text:
        messagebox.showwarning(
            "Warning",
            "Enter a message and signature."
        )
        return

    try:
        signature = int(signature_text)
    except ValueError:
        messagebox.showerror(
            "Error",
            "Invalid signature."
        )
        return

    e, n = public_key

    received_hash = pow(signature, e, n)

    hash_value, current_hash = get_hash(message, n)

    if received_hash == current_hash:
```

```
*cs_prac3_gui.py - C:/Users/info/OneDrive/Desktop/cs_prac3_gui.py (3.14.6)
File Edit Format Run Options Window Help

        text="✓ SIGNATURE VALID\n\n"
        "Message is authentic.\n"
        "Message integrity is verified.",
        fg="green"
    )

    else:

        result_label.config(
            text="X SIGNATURE INVALID\n\n"
            "Message may have been modified.\n"
            "Integrity verification failed.",
            fg="red"
        )

def clear_all():
    message_text.delete("1.0", tk.END)

    hash_entry.delete(0, tk.END)

    signature_entry.delete(0, tk.END)

    result_label.config(
        text="Verification result will appear here.",
        fg="black"
    )

    status_label.config(
        text="Ready",
        fg="black"
    )

#GUI
root = tk.Tk()
root.title("RSA Digital Signature")
root.geometry("850x700")
root.resizable(False, False)
```

SHETH L.U.J AND SIR M.V. COLLEGE

SUBJECT:CYBER & INFORMATION SECURITY

```
cs_prac3_gui.py - C:/Users/info/OneDrive/Desktop/cs_prac3_gui.py (3.14.6)
File Edit Format Run Options Window Help

# Header
header = tk.Frame(root, bg="#1f2937", height=80)
header.pack(fill="x")
title = tk.Label(
    header,
    text="RSA DIGITAL SIGNATURE",
    font=("Arial", 24, "bold"),
    bg="#1f2937",
    fg="white"
)
title.pack(pady=20)

subtitle = tk.Label(
    root,
    text="Cyber & Information Security | MU NEP 2020 - Semester 5",
    font=("Arial", 11),
    fg="#555555"
)
subtitle.pack(pady=10)

# Key section
key_frame = tk.LabelFrame(
    root,
    text="RSA Key Generation",
    font=("Arial", 11, "bold"),
    padx=15,
    pady=10
)
key_frame.pack(fill="x", padx=25, pady=5)

generate_button = tk.Button(
    key_frame,
    text="GENERATE RSA KEYS",
    command=generate_keys,
    font=("Arial", 11, "bold"),
    padx=15,
    pady=8
)

Ln: 163 Col: 21
```

```
cs_prac3_gui.py - C:/Users/info/OneDrive/Desktop/cs_prac3_gui.py (3.14.6)
File Edit Format Run Options Window Help

public_key_label = tk.Label(
    key_frame,
    text="Public Key: Not generated",
    anchor="w",
    justify="left",
    font=("Consolas", 9)
)
public_key_label.pack(fill="x", pady=5)

private_key_label = tk.Label(
    key_frame,
    text="Private Key: Not generated",
    anchor="w",
    justify="left",
    font=("Consolas", 9)
)
private_key_label.pack(fill="x")

# Message section
message_frame = tk.LabelFrame(
    root,
    text="Message",
    font=("Arial", 11, "bold"),
    padx=10,
    pady=10
)
message_frame.pack(fill="x", padx=25, pady=10)

message_text = tk.Text(
    message_frame,
    height=4,
    font=("Arial", 11),
    wrap="word"
)
message_text.pack(fill="x")

Ln: 163 Col: 21
```

SHETH L.U.J AND SIR M.V. COLLEGE

SUBJECT:CYBER & INFORMATION SECURITY

```
cs_prac3_gui.py - C:/Users/info/OneDrive/Desktop/cs_prac3_gui.py (3.14.6)
File Edit Format Run Options Window Help
)
message_text.pack(fill="x")

# Hash
hash_frame = tk.Frame(root)
hash_frame.pack(fill="x", padx=25, pady=3)

tk.Label(
    hash_frame,
    text="SHA-256 Hash:",
    font=("Arial", 10, "bold")
).pack(anchor="w")

hash_entry = tk.Entry(
    hash_frame,
    font=("Consolas", 9)
)
hash_entry.pack(fill="x", pady=3)

# Signature
signature_frame = tk.Frame(root)
signature_frame.pack(fill="x", padx=25, pady=3)

tk.Label(
    signature_frame,
    text="Digital Signature:",
    font=("Arial", 10, "bold")
).pack(anchor="w")

signature_entry = tk.Entry(
    signature_frame,
    font=("Consolas", 10)
)
signature_entry.pack(fill="x", pady=3)

# Buttons
Ln: 163 Col: 21
```

```
cs_prac3_gui.py - C:/Users/info/OneDrive/Desktop/cs_prac3_gui.py (3.14.6)
File Edit Format Run Options Window Help
signature_entry.pack(fill="x", pady=3)

# Buttons
button_frame = tk.Frame(root)
button_frame.pack(pady=15)

sign_button = tk.Button(
    button_frame,
    text="SIGN MESSAGE",
    command=sign_message,
    font=("Arial", 11, "bold"),
    padx=20,
    pady=8
)
sign_button.grid(row=0, column=0, padx=10)

verify_button = tk.Button(
    button_frame,
    text="VERIFY SIGNATURE",
    command=verify_signature,
    font=("Arial", 11, "bold"),
    padx=20,
    pady=8
)
verify_button.grid(row=0, column=1, padx=10)

clear_button = tk.Button(
    button_frame,
    text="CLEAR",
    command=clear_all,
    font=("Arial", 11, "bold"),
    padx=25,
    pady=8
)
clear_button.grid(row=0, column=2, padx=10)

# Result
Ln: 163 Col: 21
```

SHETH L.U.J AND SIR M.V. COLLEGE

SUBJECT: CYBER & INFORMATION SECURITY

```
cs_pract_gui.py - C:/Users/info/OneDrive/Desktop/cs_pract_gui.py (3.14.6)
File Edit Format Run Options Window Help

)
clear_button.grid(row=0, column=2, padx=10)

# Result
result_frame = tk.LabelFrame(
    root,
    text="Verification Result",
    font=("Arial", 11, "bold"),
    padx=10,
    pady=10
)
result_frame.pack(fill="x", padx=25, pady=5)

result_label = tk.Label(
    result_frame,
    text="Verification result will appear here.",
    font=("Arial", 12, "bold"),
    justify="center"
)
result_label.pack(pady=10)

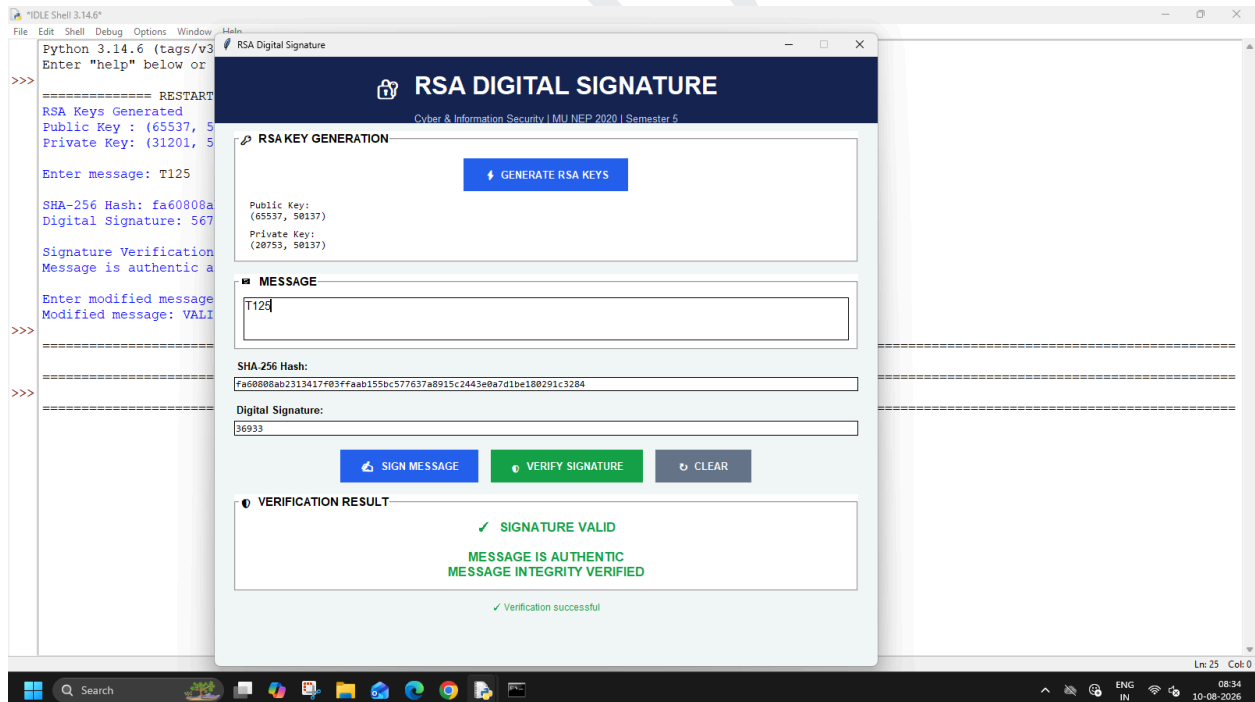
# Status
status_label = tk.Label(
    root,
    text="Ready",
    font=("Arial", 10)
)
status_label.pack(pady=8)

# Variables
public_key = None
private_key = None

root.mainloop()

Ln: 163 Col: 21
```

OUTPUT:



SUMEET YADAV
T125